

Uninformed Search in Python – Part 1

Introduction

In computer science, **problem-solving** refers to artificial intelligence techniques, including various techniques such as forming efficient algorithms, heuristics, and performing root cause analysis to find desirable solutions. The basic crux of artificial intelligence is to solve problems just like humans.

Examples of Problems in Artificial Intelligence

In today's fast-paced digitized world, artificial intelligence techniques are used widely to automate systems that can use the resource and time efficiently. Some of the well-known problems experienced in everyday life are games and puzzles. Using AI techniques, we can solve these problems efficiently. In this sense, some of the most common problems resolved by AI are:

- Travelling Salesman Problem
- Tower of Hanoi Problem
- Water-Jug Problem
- N-Queen Problem
- Chess
- Sudoku
- Crypt-arithmetic Problems
- Magic Squares
- Logical Puzzles and so on.

Problem Solving Techniques

In artificial intelligence, problems can be solved by using searching algorithms, evolutionary computations, knowledge representations, etc. In this workbook, I am going to discuss the various searching techniques that are used to solve a problem. In general, **searching** is referred to as finding information one needs.

The process of problem-solving using searching consists of the following steps.

- Define the problem
- Analyse the problem
- Identification of possible solutions
- Choosing the optimal solution
- Implementation

Let's discuss some of the essential properties of search algorithms.

Properties of search algorithms

Completeness

A search algorithm is said to be complete when it gives a solution or returns any solution for a given random input.

Optimality

If a solution found is best (lowest path cost) among all the solutions identified, then that solution is said to be an optimal one.

Time complexity

The time taken by an algorithm to complete its task is called time complexity. If the algorithm completes a task in a lesser amount of time, then it is an efficient one.

Space complexity

It is the maximum storage or memory taken by the algorithm at any time while searching.

These properties are also used to compare the efficiency of the different types of searching algorithms.

Types of search algorithms

Now let's see the types of the search algorithm. Based on the search problems, we can classify the search algorithm as

- Uninformed search
- Informed search

Uninformed search algorithms

Whoever it may be (humans or AI) need to think of all possible ways to reach the goal state (if it exists) from the initial state, all the consequences, etc. Similarly, AI systems use various search algorithms for a particular goal state (if it exists). As the name '**Uninformed Search**' means the machine blindly follows the algorithm regardless of whether right or wrong, efficient, or in-efficient. These algorithms are brute force operations, and they don't have extra information about the search space; the only information they have is on how to traverse or visit the nodes in the tree and how to find the goal state. Thus, **uninformed search algorithms** are also called **blind search algorithms**. The search algorithm produces the search tree without using any domain knowledge, which is a brute force in nature. They don't have any background information on how to approach the goal or whatsoever. But these are the basics of search algorithms in AI.

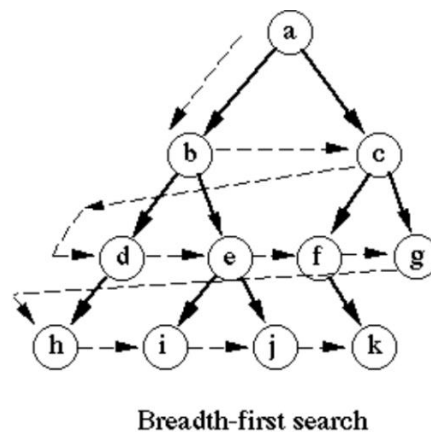
The uninformed search strategies are of the following six types:

- Breadth-first search (BFS)
- Depth-first search (DFS)
- Depth-limited search (DLS)
- Iterative deepening depth-first search (IDDFS)
- Bidirectional search (BS)
- Uniform cost search (UCS)

Let's discuss these six strategies one by one.

1. Breadth-first search (BFS)

It is one of the most common search strategies. It generally starts from the root node and examines the neighbour nodes and then moves to the next level. It uses First-in First-out (FIFO) strategy as it gives the shortest path to achieving the solution. Like DFS, it is another search algorithm in AI which traverses breadthwise to search the goal in a tree. It begins searching from the root node and expands the successor node before going expanding it further expands along breadthwise and traverses those nodes rather than searching depth-wise. BFS is used where the given problem is very small and space complexity is not considered.



The above figure is an example of a BFS Algorithm. It starts from the root node A and then traverses node B. Till this step it is the same as DFS. But here instead of expanding the children of B as in the case of DFS we expand the other child of A i.e., node C because of BFS and then move to the next level and traverse from D to G and then from H to K in this typical example. To traverse here we have only taken into consideration the lexicographical order. This is how the BFS Algorithm is implemented.

The Breadth-first search algorithm is an algorithm used to solve the shortest path problem in a graph without edge weights (i.e., a graph where all nodes are the same "distance" from each other, and they are either connected or not). This means that given a number of nodes and the edges between them, the Breadth-first search algorithm finds the shortest path from the specified start node to all other nodes. Nodes are sometimes referred to as vertices (plural of vertex) - here, we'll call them nodes.

Description of the Algorithm

The basic principle behind the Breadth-first search algorithm is to take the current node (the start node in the beginning) and then add all its neighbours that we haven't visited yet to a queue. Continue this with the next node in the queue (in a queue that is the "oldest" node). Before we add a node to the queue, we set its distance to the distance of the current node plus 1 (since all edges are weighted equally), with the distance to the start node being 0. This is repeated until there are no more nodes in the queue (all nodes are visited).

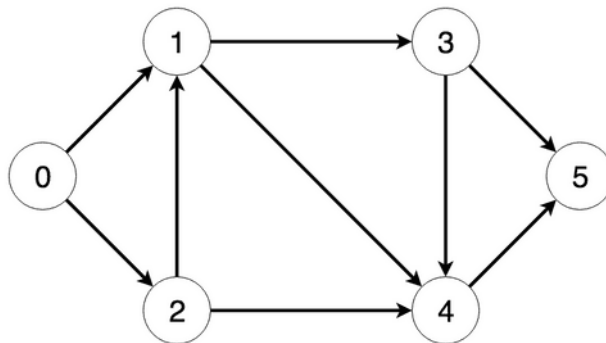
In more detail, this leads to the following Steps:

1. Initialize the distance to the starting node as 0. The distances to all other nodes do not need to be initialized since every node is visited exactly once.
2. Set all nodes to "unvisited"
3. Add the first node to the queue and label it visited.
4. While there are nodes in the queue:
 1. Take a node out of the queue
 2. For all nodes next to it that we haven't visited yet, add them to the queue, set their distance to the distance to the current node plus 1, and set them as "visited"

In the end, the distances to all nodes will be correct.

Example of the Algorithm

Consider the following graph:



The steps the algorithm performs on this graph if given node 0 as a starting point, in order, are:

1. Visiting node 0
2. Visited nodes: [true, false, false, false, false, false], Distances: [0, 0, 0, 0, 0, 0]
 1. Removing node 0 from the queue...
 2. Visiting node 1, setting its distance to 1 and adding it to the queue
 3. Visiting node 2, setting its distance to 1 and adding it to the queue
3. Visited nodes: [true, true, true, false, false, false], Distances: [0, 1, 1, 0, 0, 0]
 1. Removing node 1 from the queue...
 2. Visiting node 3, setting its distance to 2 and adding it to the queue
 3. Visiting node 4, setting its distance to 2 and adding it to the queue

4. Visited nodes: [true, true, true, true, true, false], Distances: [0, 1, 1, 2, 2, 0]
 1. Removing node 2 from the queue...
 2. No adjacent, unvisited nodes
5. Visited nodes: [true, true, true, true, true, false], Distances: [0, 1, 1, 2, 2, 0]
 1. Removing node 3 from the queue...
 2. Visiting node 5, setting its distance to 3 and adding it to the queue
6. Visited nodes: [true, true, true, true, true, true], Distances: [0, 1, 1, 2, 2, 3]
 1. Removing node 4 from the queue...
 2. No adjacent, unvisited nodes
7. Visited nodes: [true, true, true, true, true, true], Distances: [0, 1, 1, 2, 2, 3]
 1. Removing node 5 from the queue...
8. No more nodes in the queue. Final distances: [0, 1, 1, 2, 2, 3]

Runtime Complexity of the Algorithm

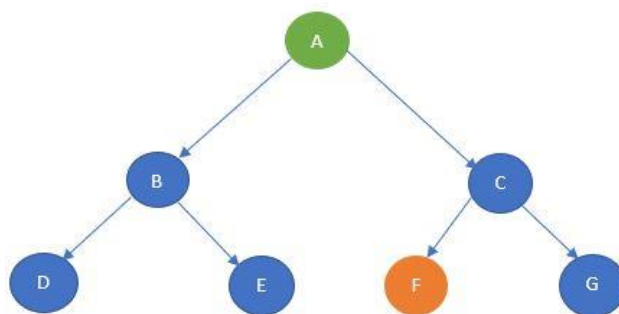
The runtime complexity of Breadth-first search is $O(|E| + |V|)$ ($|V|$ = number of Nodes, $|E|$ = number of Edges) if adjacency-lists are used. If we simply search all nodes to find connected nodes in each step and use a matrix to look up whether two nodes are adjacent, the runtime complexity increases to $O(|V|^2)$.

Depending on the graph this might not matter, since the number of edges can be as big as $|V|^2$ if all nodes are connected with each other.

Space Complexity of the Algorithm

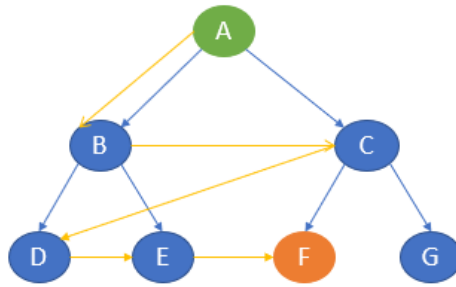
The space complexity of Breadth-first search depends on how it is implemented as well and is equal to the runtime complexity.

Now, let us consider the following tree.



Here, let's take node A as the start state and node F as the goal state. The BFS algorithm starts with the start state and then goes to the next level and visits the node until it reaches the goal state.

In this example, it starts from A and then travel to the next level and visits B and C and then travel to the next level and visits D, E, F and G. Here, the goal state is defined as F. So, the traversal will stop at F.



The path of traversal is:

A → B → C → D → E → F

Implementation in Python

The following is an implementation of BFS using the adjacency matrix. First, I'm including a template (pseudocode) and then the concrete implementation in Python.

```

def bfs(graph, start):
    """
    Implementation of Breadth-First-Search (BFS) using adjacency matrix.
    This returns nothing (yet), it is meant to be a template for whatever you want to
    do with it,
    e.g. finding the shortest path in a unweighted graph.
    This has a runtime of  $O(|V|^2)$  ( $|V|$  = number of Nodes), for a faster implementation
    see @see ../fast/BFS.java (using adjacency Lists)
    :param graph: an adjacency-matrix-representation of the graph where (x,y) is True
    if the the there is an edge between nodes x and y.
    :param start: the node to start from.
    :return: Array array containing the shortest distances from the given start node to
    each other node
    """
    # A Queue to manage the nodes that have yet to be visited, intialized with the
    start node
    queue = [start]
    # A boolean array indicating whether we have already visited a node
    visited = [False] * len(graph)
    # (The start node is already visited)
    visited[start] = True
    # Keeping the distances (might not be necessary depending on your use case)
    distances = [float("inf")] * len(
        graph) # Technically no need to set initial values since every node is visted
    exactly once
    # (the distance to the start node is 0)

```

```

distances[start] = 0
# While there are nodes left to visit...
while len(queue) > 0:
    print("Visited nodes: " + str(visited))
    print("Distances: " + str(distances))
    node = queue.pop(0)
    print("Removing node " + str(node) + " from the queue...")
    # ...for all neighboring nodes that haven't been visited yet....
    for i in range(len(graph[node])):
        if graph[node][i] and not visited[i]:
            # Do whatever you want to do with the node here.
            # Visit it, set the distance and add it to the queue
            visited[i] = True
            distances[i] = distances[node] + 1
            queue.append(i)
            print("Visiting node " + str(i) + ", setting its distance to " + str(
                distances[i]) + " and adding it to the queue")

    print("No more nodes in the queue. Distances: " + str(distances))
return distances

```

Let's implement BFS in Python.

sample graph implemented as a dictionary - The illustrated graph is represented using an adjacency list. An easy way to do this in Python is to use a dictionary data structure, where each vertex has a stored list of its adjacent nodes.

```

graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['F'],
    'F' : []
}

```

```

visited = [] # visited is a list to keep track of visited nodes.

```

```

queue = [] #Initialize a queue - queue is a list that is used to keep track of nodes
currently in the queue

# The arguments of the following bfs function are the visited list, the graph in the
form of a dictionary, and the starting node A. bfs follows the algorithm described
above - 1. It checks and appends the starting node to the visited list and the queue.
2. Then, while the queue contains elements, it keeps taking out nodes from the queue,
appends the neighbours of that node to the queue if they are unvisited, and marks them
as visited. 3. This continues until the queue is empty.

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        s = queue.pop(0)
        print (s, end = " ")

        for neighbour in graph[s]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
bfs(visited, graph, 'A')

```

The output path generated is as follows.

A B C D E F

Regarding the time complexity, since all of the nodes and vertices are visited, the time complexity for BFS on a graph is $O(V+E)$; where V is the number of vertices and E is the number of edges.

Advantages:

- BFS will provide a solution if any solution exists.
- If there is more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.
- BFS will never be trapped in any unwanted nodes.

- If the graph has more than one solution, then BFS will return the optimal solution which provides the shortest path.

Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Conclusions

It requires a lot of memory space and time consuming if the goal state is at the bottom or end. It uses a FIFO queue DS to implement.

Complete: Yes (assuming b is finite)

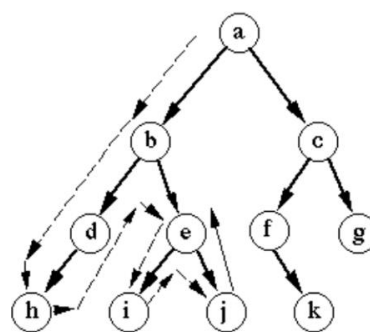
Time Complexity: $O(b^d)$

Space complexity: $O(b^d)$

Optimal: Yes, if Step cost = 1 (i.e., no cost/all step costs are same)

2. Depth-first search (DFS)

It is a search algorithm where the search tree will be traversed from the root node. It will be traversing, searching for a key at the leaf of a particular branch. If the key is not found the searching retraces its steps back to the point from where the other branch was left unexplored, and the same procedure is repeated for that other branch.



Depth-first search

The above image clearly explains the DFS Algorithm. First, the searching starts in the root node A and then goes to the branch where node B is present (lexicographical order). Then it goes to node D because of DFS and from D there is the only node to traverse i.e., node H. But after node H does not have any child nodes so we retrace the path in which we traversed earlier and again reach node B but this time we traverse through

in the untraced path a traverse through node E. There are two branches at node E but let's traverse node I (lexicographical order) and then retrace the path as we have no further nodes after E to traverse. Then we traverse node J as it is the untraced branch and then again, we are at the end and retrace the path and reach node B and then we will traverse the untraced branch i.e., through node C and repeat the same process. This is called the DFS Algorithm.

The Depth-First Search (also DFS) algorithm is an algorithm used to find a node in a tree. This means that given a tree data structure, the algorithm will return the first node in this tree that matches the specified condition (i.e., being equal to a value). Nodes are sometimes referred to as vertices (plural of vertex) - here, we'll call them nodes. The edges must be unweighted. This algorithm can also work with unweighted graphs if a mechanism to keep track of already visited nodes is added.

Description of the Algorithm

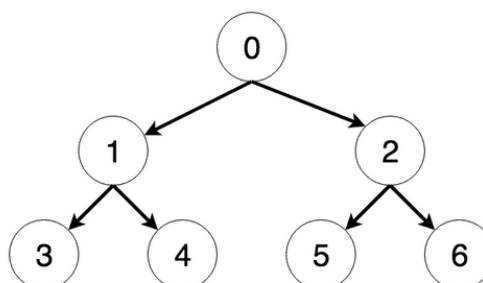
The basic principle of the algorithm is to start with a start node, and then look at the first child of this node. It then looks at the first child of that node (grandchild of the start node) and so on, until a node has no more children (we've reached a leaf node). It then goes up one level and looks at the next child. If there are no more children, it goes up one more level, and so on, until it finds more children or reaches the start node. If hasn't found the goal node after returning from the last child of the start node, the goal node cannot be found, since by then all nodes have been traversed.

Specifically, these are the steps:

1. For each child of the current node
2. If it is the target node, return. The node has been found.
3. Set the current node to this node and go back to 1.
4. If there are no more child nodes to visit, return to the parent.
5. If the node has no parent (i.e., it is the root), return. The node has not been found.

Example of the Algorithm

Consider the following tree:



The steps the algorithm performs on this tree if given node 0 as a starting point, in order, are:

1. Visiting Node 0

2. Visiting Node 1
3. Visiting Node 3
4. Went through all children of 3, returning to its parent.
5. Visiting Node 4
6. Went through all children of 4, returning to its parent.
7. Went through all children of 1, returning to its parent.
8. Visiting Node 2
9. Visiting Node 5
10. Went through all children of 5, returning to its parent.
11. Visiting Node 6
12. Found the node we're looking for!

Runtime of the Algorithm

The runtime of regular Depth-First Search (DFS) is $O(|N|)$ ($|N|$ = number of Nodes in the tree), since every node is traversed at most once. The number of nodes is equal to b^d , where b is the branching factor and d is the depth, so the runtime can be rewritten as $O(b^d)$.

Space of the Algorithm

The space complexity of Depth-First Search (DFS) is, if we exclude the tree itself, $O(d)$, with d being the depth, which is also the size of the call stack at maximum depth. If we include the tree, the space complexity is the same as the runtime complexity, as each node needs to be saved.

Implementation in Python

The following is an implementation of DFS in Python. First, I'm including a template (pseudocode) and then the concrete implementation in Python.

```
def dfs(start, target):  
    """  
    Implementation of DFS (depth-first search) algorithm to find the shortest path from  
    a start to a target node..  
    Given a start node, this returns the node in the tree below the start node with the  
    target value (or null if it doesn't exist)  
    Runs in  $O(n)$ , where  $n$  is the number of nodes in the tree, or  $O(b^d)$ , where  $b$  is the  
    branching factor and  $d$  is the depth.  
    :param start: the node to start the search from  
    :param target: the value to search for  
    :return: The node containing the target value or null if it doesn't exist.  
    """  
    print("Visiting Node " + str(start["value"]))  
    if start["value"] == target:  
        # We have found the goal node we we're searching for
```

```

    print("Found the node we're looking for!")
    return start

# Recurse with all children
for i in range(len(start["children"])):
    result = dfs(start["children"][i], target)
    if result is not None:
        # We've found the goal node while going down that child
        return result

# We've gone through all children and not found the goal node
print("Went through all children of " + str(start["value"]) + ", returning to it's parent.")
return None

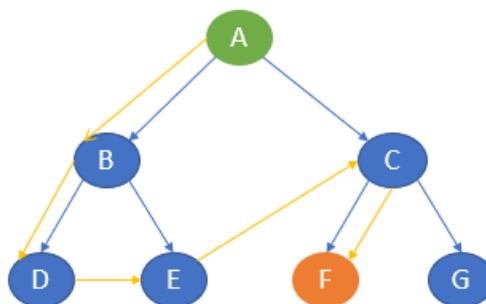
```

The depth-first search uses Last-in, First-out (LIFO) strategy and hence it can be implemented by using stack. DFS uses backtracking. That is, it starts from the initial state and explores each path to its greatest depth before it moves to the next path.

DFS will follow: Root node → Left node → Right node

Now, consider the same example tree mentioned above.

Here, it starts from the start state A and then travels to B and then it goes to D. After reaching D, it backtracks to B. B is already visited, hence it goes to the next depth E and then backtracks to B. as it is already visited, it goes back to A. A is already visited. So, it goes to C and then to F. F is our goal state and it stops there.



The path of traversal is:

A → B → D → E → C → F

#Let's try to code it.

```

# Using a Python dictionary to act as an adjacency list
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F', 'G'],
    'D' : [],
    'E' : [],
    'F' : [],
    'G' : []
}

goal = 'F'

visited = set() # Set to keep track of visited nodes.

# The dfs function is called and is passed the visited set, the graph in the form of a
# dictionary, and A, which is the starting node.

# dfs follows the algorithm described above: 1. It first checks if the current node is
# unvisited - if yes, it is appended in the visited set. 2. Then for each neighbor of the
# current node, the dfs function is invoked again. 3. The base case is invoked when all
# the nodes are visited. The function then returns.

def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            if goal in visited:
                break
            else:
                dfs(visited, graph, neighbour)
dfs(visited, graph, 'A')

```

The output path is as follows.

```

A
B
D
E
C
F

```

Regarding the time complexity, since all the nodes and vertices are visited, the average time complexity for DFS on a graph is $O(V + E)$, where V is the number of vertices and E is the number of edges. In case of DFS on a tree, the time complexity is $O(V)$, where V is the number of nodes.

Note. We say average time complexity because a set's in operation has an average time complexity of $O(1)$. If we used a list, the complexity would be higher.

Advantages:

- DFS requires lesser memory as compared to BFS, as it only needs to store a stack of the nodes on the path from the root node to the current node.
- It takes less time to reach the goal node than the BFS algorithm (if it traverses in the right path).
- DFS does not require much more search.

Disadvantages:

- There is the possibility that many states keep reoccurring, and there is no guarantee of finding the solution. Thus, DFS does not always guarantee to give a solution.
- The DFS algorithm goes for deep down searching and sometimes it may get trapped in an infinite loop.

Conclusions

It occupies a lot of memory space, and time to execute when the solution is at the bottom or end of the tree and is implemented using LIFO Stack data structure [DS].

Complete: No

Time Complexity: $O(b^m)$

Space complexity: $O(bm)$

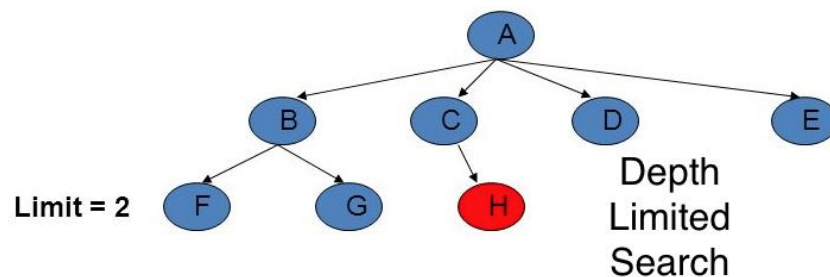
Optimal: Yes

3. Depth-limited search (DLS)

Depth-limited works similarly to depth-first search. The difference here is that depth-limited search has a pre-defined limit up to which it can traverse the nodes. Depth-limited search solves one of the drawbacks of DFS as it does not go to an infinite path. In another words, the sad failure of DFS is alleviated by supplying a depth-first search with a predetermined depth limit. That is, nodes at depth are treated as if they have no successors. This approach is called a **depth-limited search**. The depth limit solves the infinite-path problem.

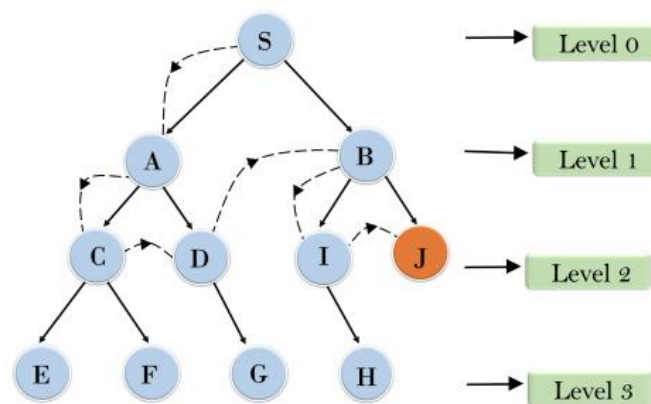
DLS ends its traversal if any of the following conditions exits.

- **Standard Failure** - It denotes that the given problem does not have any solutions.
- **Cut off Failure Value** - It indicates that there is no solution for the problem within the given depth-limit.



The above figure illustrates the implementation of the DLS algorithm. Node A is at Limit = 0, followed by nodes B, C, D, and E at Limit = 1 and nodes F, G, and H at Limit = 2. Our start state is considered to be node A and our goal state is node H. To reach node H we apply DLS. So, in the first case let's, set our limit to 0 and search for the goal.

Since limit 0, the algorithm will assume that there are no children after limit 0 even if nodes exist further. Now, if we implement it, we will traverse only node A as there is only one node in limit 0, which is basically our goal state. If we use SFV, it says that there is no solution to the problem at limit 0 whereas LCV says that there is no solution for the problem until the set depth limit. Now since we were not able to find the goal, we increase our limit to 1 and apply DFS till limit 1 even though there are further nodes after limit 1. But those nodes aren't expanded as we have set our limit as 1. Hence nodes A followed by B, C, D, and E are expanded as the mentioned order. As in our first case, if we use SFV, it says that there is no solution to the problem at limit 1 whereas LCV says that there is no solution for the problem until the set depth limit 1. Hence, we again increase our limit from 1 to 2 in the notion to find the goal. Till limit 2 DFS will be implemented from our start node A and its children B, C, D, and E and then from E it moves to F similarly backtracks the path and explores the unexplored branch where node G is present and then retraces the path and explores the child of C i.e., node H and then we finally we reach our goal by applying DLS Algorithm. Suppose we have further successors of node F but only the nodes till the limit 2 will be explored as we have limited the depth and have reached the goal state.



This image explains the DLS implementation and could be referred to for better understanding.

Depth-limited search can be terminated with two Conditions of failure:

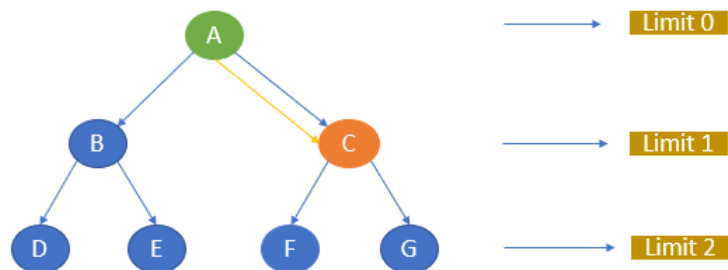
Standard Failure: it indicates that the problem does not have any solutions.

Cut-off Failure Value: It defines no solution for the problem within a given depth limit.

Now, let us consider the same example.

Let's take A as the start node and C as the goal state and limit as 1.

The traversal first starts with node A and then goes to the next level 1 and the goal state C is there. It stops the traversal.



The path of traversal is A → C

If we give C as the goal node and the limit as 0, the algorithm will not return any path as the goal node is not available within the given limit. If we give the goal node as F and limit as 2, the path will be A, C, F.

Implementation in Python

Let's implement DLS in Python.

```
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F', 'G'],
    'D' : [],
    'E' : [],
    'F' : [],
    'G' : []
}

def DLS(start,goal,path,level,maxD):
    print('\nCurrent level-->',level)
```

```

    path.append(start)
    if start == goal:
        print("Goal test successful")
        return path
    print('Goal node testing failed')
    if level==maxD:
        return False
    print('nExpanding the current node',start)
    for child in graph[start]:
        if DLS(child,goal,path,level+1,maxD):
            return path
        path.pop()
    return False

start = 'A'
goal = input('Enter the goal node:-')
maxD = int(input("Enter the maximum depth limit:-"))
print()
path = list()
res = DLS(start,goal,path,0,maxD)
if(res):
    print("Path to goal node available")
    print("Path",path)
else:
    print("No path available for the goal node in given depth limit")

```

When we give C as goal node and 1 as limit the path will be as follows.

```

Enter the goal node:-C
Enter the maximum depth limit:-1

nCurrent level--> 0
Goal node testing failed
nExpanding the current node A
nCurrent level--> 1
Goal node testing failed
nCurrent level--> 1
Goal test successful
Path to goal node available
Path ['A', 'C']

```

Advantages:

Depth-limited search is Memory efficient. It takes lesser memory when compared to other search techniques.

Disadvantages:

The DLS has disadvantages of completeness and is not optimal if it has more than one goal state.

Conclusions:

Complete: Complete (if solution > depth-limit)

Time Complexity: $O(b^l)$ where, $l \rightarrow$ depth-limit

Space complexity: $O(bl)$

Optimal: Yes (only if $l > d$)